

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №5
по курсу «Проектирование и реализация баз данных»
Вариант 10 “Автовокзал”

Выполнил:

Гашимов И.Ф.

К3239

Проверила:

Говорова М.М.

Санкт-Петербург

2026 г.

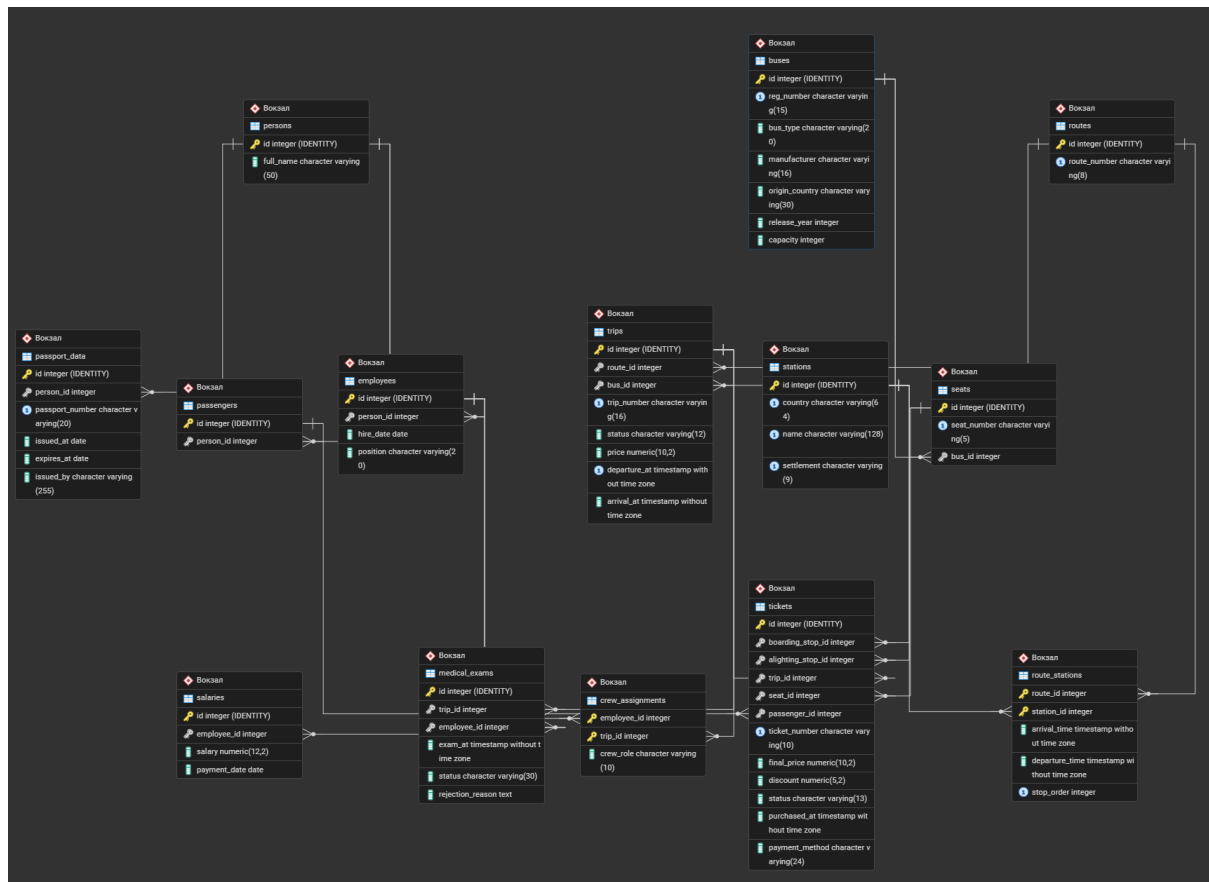
Цель работы: овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

В рамках лабораторной работы необходимо создать 3 процедуры для индивидуальной базы данных согласно варианту, а также создать триггеры для контроля бизнес-правил предметной области.

Для варианта 10 “Автовокзал” реализуются процедуры:

1. продажа билета;
2. возврат билета;
3. добавление нового рейса.

Для контроля целостности данных создаются 7 триггеров по ИЗ.



ERD

Подключаемся в PSQL к таблице Vokzal

1. Создание процедур

1 Процедура - Продажи билета

```
CREATE OR REPLACE PROCEDURE "Vokzal".sell_ticket(
    IN p_trip_id integer,
    IN p_passenger_id integer,
    IN p_boarding_stop_id integer,
    IN p_dropoff_stop_id integer,
    IN p_seat_id integer,
    IN p_purchase_method varchar,
    IN p_ticket_number varchar
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_base_price numeric(10,2);
    v_final_price numeric(10,2);
    v_discount numeric(5,2);
BEGIN
    -- Получаем базовую цену рейса
    SELECT "price"
    INTO v_base_price
    FROM "Vokzal"."trip"
    WHERE "trip_id" = p_trip_id;

    IF v_base_price IS NULL THEN
        RAISE EXCEPTION 'Рейс с trip_id = % не найден или цена рейса не задана',
        p_trip_id;
    END IF;

    -- Если место не указано, билет без места: скидка 10%
    IF p_seat_id IS NULL THEN
        v_discount := 10;
        v_final_price := ROUND(v_base_price * 0.9, 2);
    ELSE
        v_discount := 0;
        v_final_price := v_base_price;
    END IF;

    -- Добавление билета
    INSERT INTO "Vokzal"."ticket" (
        "boarding_stop_id",
        "dropoff_stop_id",
        "trip_id",
        "seat_id",
        "passenger_id",
        "number",
        "price",
```

```

        "discount",
        "status",
        "purchase_date",
        "purchase_method"
    )
VALUES (
    p_boarding_stop_id,
    p_dropoff_stop_id,
    p_trip_id,
    p_seat_id,
    p_passenger_id,
    p_ticket_number,
    v_final_price,
    v_discount,
    'Куплен',
    NOW(),
    p_purchase_method
);

    RAISE NOTICE 'Билет % успешно продан. Итоговая цена: %', p_ticket_number,
v_final_price;
END;
$$;

```

Видим что процедура создалась успешно:

```

Vokzal$#      'Куплен',
Vokzal$#      NOW(),
Vokzal$#      p_purchase_method
Vokzal$#      );
Vokzal$#
Vokzal$#      RAISE NOTICE 'Билет % успешно продан. Итоговая цена: %', p_tick
et_number, v_final_price;
Vokzal$# END;
Vokzal$# $$;
CREATE PROCEDURE
Vokzal=#

```

Что делает:

- принимает рейс, пассажира, остановку посадки, остановку высадки, место, способ оплаты;
- создаёт билет;
- если место не указано, билет считается билетом без места;
- цена берётся из рейса;
- для билета без места цена должна быть снижена на 10%;
- статус нового билета Куплен.

Посмотрим список рейсов для выбора тестового рейса:

```
Vokzal=# price,
Vokzal=# departure_date,
Vokzal=# arrival_date
Vokzal=# FROM "Vokzal"."trip"
Vokzal=# ORDER BY trip_id
Vokzal=# LIMIT 10;
```

trip_id	route_id	bus_id	number	status	price	departure_date	arrival_date
1	1	1	REIS-1001	Прибыл	2045.00	2026-04-11 08:30:00	2026-04-11 13:30:00
2	2	2	REIS-1002	Прибыл	3141.00	2026-04-10 09:45:00	2026-04-10 13:45:00
3	3	3	REIS-1003	Прибыл	1835.00	2026-04-09 10:30:00	2026-04-09 12:30:00
4	4	4	REIS-1004	Прибыл	1232.00	2026-04-08 11:30:00	2026-04-08 15:30:00
5	5	6	REIS-1005	Прибыл	1726.00	2026-04-07 12:00:00	2026-04-07 22:00:00
6	6	7	REIS-1006	Прибыл	1150.00	2026-04-06 13:15:00	2026-04-06 17:15:00
7	7	9	REIS-1007	Прибыл	1239.00	2026-04-05 14:30:00	2026-04-05 16:30:00
8	8	1	REIS-1008	Прибыл	2167.00	2026-04-04 15:45:00	2026-04-04 17:45:00
9	9	2	REIS-1009	Прибыл	3111.00	2026-04-03 08:45:00	2026-04-03 13:45:00
10	10	3	REIS-1010	Прибыл	764.00	2026-04-02 09:30:00	2026-04-02 16:30:00

(10 строк)

Выберем рейс с trip_id = 4, где его route_id = 4, bus_id = 4, price = 1232.0

Посмотрим на остановки выбранного маршрута:

```
Vokzal=# SELECT
Vokzal=#     sir.route_id,
Vokzal=#     sir.stop_id,
Vokzal=#     s.name
Vokzal=# FROM "Vokzal"."stop_in_route" sir
Vokzal=# JOIN "Vokzal"."stop" s
Vokzal=#     ON s.stop_id = sir.stop_id
Vokzal=# WHERE sir.route_id = 4
Vokzal=# ORDER BY sir.stop_id;
```

route_id	stop_id	name
4	9	Минск, остановка 9
4	10	Смоленск, остановка 10
4	15	Великий Новгород, остановка 15

(3 строк)

Выберем 2 остановки (stop_id) и обозначим:

- boarding_stop_id = 9
- dropoff_stop_id = 15

Так же посмотрим на пассажиров:

```
Vokzal=# SELECT
Vokzal=#     passenger_id,
Vokzal=#     person_id
Vokzal=# FROM "Vokzal"."passenger"
Vokzal=# ORDER BY passenger_id
Vokzal=# LIMIT 10;
```

passenger_id	person_id
1	128
2	35
3	70
4	46
5	83
6	43
7	23
8	18
9	50
10	121

(10 строк)

Выберем допустим passenger_id = 8

Смотрим список мест выбранного автобуса:

```
Vokzal=# SELECT
Vokzal-#      seat_id,
Vokzal-#      bus_id,
Vokzal-#      number
Vokzal-# FROM "Vokzal"."seat"
Vokzal-# WHERE bus_id = 4
Vokzal-# ORDER BY seat_id
Vokzal-# LIMIT 10;
 seat_id | bus_id | number
-----+-----+-----
    116 |      4 |      1
    117 |      4 |      2
    118 |      4 |      3
    119 |      4 |      4
    120 |      4 |      5
    121 |      4 |      6
    122 |      4 |      7
    123 |      4 |      8
    124 |      4 |      9
    125 |      4 |     10
```

Выбираем допустим seat_id = 123

Посмотрим на содержание таблицы ticket до продажи нового билета:

```
Vokzal=# SELECT
Vokzal-#      ticket_id,
Vokzal-#      number,
Vokzal-#      trip_id,
Vokzal-#      passenger_id,
Vokzal-#      boarding_stop_id,
Vokzal-#      dropoff_stop_id,
Vokzal-#      seat_id,
Vokzal-#      price,
Vokzal-#      discount,
Vokzal-#      status,
Vokzal-#      purchase_date,
Vokzal-#      purchase_method
Vokzal-# FROM "Vokzal"."ticket"
Vokzal-# ORDER BY ticket_id DESC
Vokzal-# LIMIT 10;
 ticket_id | number | trip_id | passenger_id | boarding_stop_id | dropoff_stop_id | seat_id | price | discount | status | purchase_date | purchase_method
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
    1484 | T862564 |      80 |          5 |                29 |                16 |     145 | 915.00 |      0.00 | Исползован | 2026-04-08 08:15:00 | Онлай
    1483 | T673629 |      80 |         27 |                30 |                37 |     146 | 915.00 |      0.00 | Возвращён | 2026-04-06 06:15:00 | Онлай
    1482 | T383941 |      80 |          3 |                16 |                30 |     127 | 915.00 |      0.00 | Исползован | 2026-04-08 10:15:00 | Онлай
    1481 | T502537 |      80 |         49 |                29 |                16 |     137 | 915.00 |      0.00 | Куплен | 2026-04-07 03:15:00 | Онлай
    1480 | T857641 |      80 |         22 |                29 |                30 |     121 | 915.00 |      0.00 | Исползован | 2026-04-01 01:15:00 | О-- Д
    1479 | T674608 |      80 |         49 |                16 |                37 |     149 | 915.00 |      0.00 | Исползован | 2026-03-29 00:15:00 | Касса
    1478 | T229287 |      80 |         19 |                16 |                37 |     119 | 915.00 |      0.00 | Исползован | 2026-03-31 03:15:00 | Онлай
    1477 | T742699 |      80 |          2 |                30 |                37 |     126 | 915.00 |      0.00 | Исползован | 2026-04-03 00:15:00 | Онлай
    1476 | T634256 |      80 |         46 |                29 |                37 |     124 | 915.00 |      0.00 | Исползован | 2026-04-08 12:15:00 | Касса
    1475 | T111593 |      80 |         22 |                29 |                30 |     132 | 915.00 |      0.00 | Исползован | 2026-03-31 10:15:00 | Касса
(10 строк)
```

Теперь сделаем вызов процедуры:

```
Vokzal=# CALL "Vokzal".sell_ticket(
Vokzal(#      4,
Vokzal(#      8,
Vokzal(#      9,
Vokzal(#     15,
Vokzal(#     123,
Vokzal(#      'Онлайн',
Vokzal(#      'T1205');
ЗАМЕЧАНИЕ: Билет T1205 успешно продан. Итоговая цена: 1232.00
CALL
```

Посмотрим повторно на таблицу ticket после вызова процедуры:

```
Vokzal=# SELECT
Vokzal=# ticket_id,
Vokzal=# number,
Vokzal=# trip_id,
Vokzal=# passenger_id,
Vokzal=# boarding_stop_id,
Vokzal=# dropoff_stop_id,
Vokzal=# seat_id,
Vokzal=# price,
Vokzal=# discount,
Vokzal=# status,
Vokzal=# purchase_date,
Vokzal=# purchase_method
Vokzal=# FROM "Vokzal"."ticket"
Vokzal=# ORDER BY ticket_id DESC
Vokzal=# LIMIT 10;
```

	ticket_id	number	trip_id	passenger_id	boarding_stop_id	dropoff_stop_id	seat_id	price	discount	status	purchase_date	purchase_method
	1485	T1205	4	8	9	15	123	1232.00	0.00	Куплен	2026-05-12 19:58:04	923225
	1484	T862564	80	5	29	16	145	915.00	0.00	Использован	2026-04-08 08:15:00	
	1483	T673629	80	27	30	37	146	915.00	0.00	Возвращён	2026-04-06 06:15:00	
	1482	T303941	80	3	16	30	127	915.00	0.00	Использован	2026-04-08 10:15:00	
	1481	T502537	80	49	29	16	137	915.00	0.00	Куплен	2026-04-07 03:15:00	
	1480	T857641	80	22	29	30	121	915.00	0.00	Использован	2026-04-01 01:15:00	
	1479	T674608	80	49	16	37	149	915.00	0.00	Использован	2026-03-29 00:15:00	
	1478	T229287	80	19	16	37	119	915.00	0.00	Использован	2026-03-31 03:15:00	
	1477	T742699	80	2	30	37	126	915.00	0.00	Использован	2026-04-03 00:15:00	
	1476	T634256	80	46	29	37	124	915.00	0.00	Использован	2026-04-08 12:15:00	

(10 строк)

Для проверки процедуры **sell_ticket** был выбран существующий рейс из таблицы **trip**, пассажир из таблицы **passenger**, две остановки маршрута из таблиц **stop_in_route** и **stop**, а также место автобуса из таблицы **seat**. Перед вызовом процедуры было выведено содержимое таблицы **ticket**. После выполнения процедуры в таблице появилась новая запись со статусом Куплен, указанным номером билета и рассчитанной стоимостью.

2 Процедура - возврат билета

```
CREATE OR REPLACE PROCEDURE "Vokzal".return_ticket(
    IN p_ticket_id integer
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_status varchar;
BEGIN
    SELECT "status"
    INTO v_status
    FROM "Vokzal"."ticket"
    WHERE "ticket_id" = p_ticket_id;

    IF v_status IS NULL THEN
        RAISE EXCEPTION 'Билет с ticket_id = % не найден', p_ticket_id;
    END IF;

    IF v_status <> 'Куплен' THEN
        RAISE EXCEPTION 'Возврат невозможен. Текущий статус билета: %', v_status;
    END IF;
```

```

UPDATE "Vokzal"."ticket"
SET "status" = 'Возвращён'
WHERE "ticket_id" = p_ticket_id;

RAISE NOTICE 'Билет с ticket_id = % успешно возвращён', p_ticket_id;
END;
$$;

```

Что делает:

- принимает ticket_id;
- проверяет, что билет существует;
- разрешает вернуть только билет со статусом Куплен;
- меняет статус на Возвращён.

Посмотрим список купленных билетов перед возвратом:

```

Vokzal=# WHERE status = 'Куплен'
Vokzal=# ORDER BY ticket_id DESC
Vokzal=# LIMIT 10;

```

	ticket_id	number	status	price	discount	purchase_date
225	1485	T1205	Куплен	1232.00	0.00	2026-05-12 19:58:04.923
	1481	T502537	Куплен	915.00	0.00	2026-04-07 03:15:00
	1472	T733754	Куплен	1825.20	10.00	2026-04-05 15:15:00
	1458	T276728	Куплен	2028.00	0.00	2026-04-02 14:15:00
	1446	T550680	Куплен	2215.00	0.00	2026-03-10 04:15:00
	1437	T504848	Куплен	2096.00	0.00	2026-02-22 10:00:00
	1433	T777423	Куплен	2096.00	0.00	2026-02-22 11:00:00
	1422	T125280	Куплен	2096.00	0.00	2026-02-28 16:00:00
	1410	T111081	Куплен	2419.00	0.00	2026-03-05 07:30:00
	1403	T875643	Куплен	2419.00	0.00	2026-03-05 11:30:00

(10 строк)

Выбираем купленный билет в 1 процедуре ticket_id = 1485 для теста

Билет выглядит так до выполнения процедуры возврата:

```

Vokzal=# SELECT
Vokzal=#     ticket_id,
Vokzal=#     number,
Vokzal=#     status,
Vokzal=#     price,
Vokzal=#     discount,
Vokzal=#     purchase_date
Vokzal=# FROM "Vokzal"."ticket"
Vokzal=# WHERE ticket_id = 1485;

```

	ticket_id	number	status	price	discount	purchase_date
---	1485	T1205	Куплен	1232.00	0.00	2026-05-12 19:58:04.9232

Вызываем возврат билета 1481:

```
Vokzal=# CALL "Vokzal".return_ticket(1485);  
ЗАМЕЧАНИЕ: Билет с ticket_id = 1485 успешно возвращён  
CALL
```

Билет выглядит так после возврата:

```
Vokzal=# WHERE ticket_id = 1485;  
 ticket_id | number | status   | price  | discount | purchase_date  
-----+-----+-----+-----+-----+-----  
-----  
          | 1485  | T1205   | Возвращён | 1232.00 | 0.00 | 2026-05-12 19:58:04.9
```

Для проверки процедуры **return_ticket** был выбран билет со статусом **Куплен**. До выполнения процедуры был зафиксирован текущий статус билета. После вызова процедуры статус выбранного билета изменился на **Возвращён**, что подтверждает корректную работу процедуры возврата.

3 Процедура - Добавление нового рейса

```
CREATE OR REPLACE PROCEDURE "Vokzal".add_trip(  
    IN p_route_id integer,  
    IN p_bus_id integer,  
    IN p_trip_number varchar,  
    IN p_departure_date timestamp,  
    IN p_arrival_date timestamp,  
    IN p_price numeric  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- Проверяем корректность времени рейса  
    IF p_departure_date >= p_arrival_date THEN  
        RAISE EXCEPTION 'Дата отправления должна быть раньше даты прибытия';  
    END IF;  
  
    -- Проверяем цену  
    IF p_price < 0 THEN  
        RAISE EXCEPTION 'Цена рейса не может быть отрицательной';  
    END IF;  
  
    -- Добавляем новый рейс  
    INSERT INTO "Vokzal"."trip" (  
        "route_id",  
        "bus_id",  
        "number",  
        "status",  
        "price",
```

```

        "departure_date",
        "arrival_date"
    )
VALUES (
    p_route_id,
    p_bus_id,
    p_trip_number,
    'Запланирован',
    p_price,
    p_departure_date,
    p_arrival_date
);

RAISE NOTICE 'Новый рейс % успешно добавлен', p_trip_number;
END;
$$;

```

Что делает:

- принимает маршрут, автобус, номер рейса, дату отправления, дату прибытия, цену;
- добавляет рейс со статусом Запланирован;
- пересечения автобусов по времени контролирует триггер.

Список маршрутов для добавления нового рейса:

```

Vokzal=# SELECT
Vokzal=#     route_id,
Vokzal=#     route_number
Vokzal=# FROM "Vokzal"."route"
Vokzal=# ORDER BY route_id
Vokzal=# LIMIT 10;
 route_id | route_number
-----+-----
        1 | 263M
        2 | 210B
        3 | 993
        4 | 979B
        5 | 327A
        6 | 936B
        7 | 458B
        8 | 940A
        9 | 328
       10 | 775A

```

Выбираем допустим route_id = 7

Список автобусов для добавления нового рейса:

```

Vokzal=# ORDER BY bus_id
Vokzal=# LIMIT 10;
 bus_id | number      | type           | seat_count
-----+-----+-----+-----
        1 | TW382BB81 | Туристический | 32
        2 | HX516BB72 | Туристический | 44
        3 | ZA195BB47 | Туристический | 39
        4 | VS477BB70 | Туристический | 40
        5 | LW564BB44 | Городской     | 53
        6 | GK222BB78 | Туристический | 53
        7 | PI841BB85 | Туристический | 39
        8 | HL283BB48 | Междугородний | 31
        9 | RJ814BB26 | Туристический | 48
       10 | JP590BB66 | Городской     | 45

```

Выбираем допустим bus_id = 8

Список рейсов до добавления нового рейса:

trip_id	route_id	bus_id	number	status	price	departure_date	arrival_date
80	19	4	REIS-1080	Прибыл	915.00	2026-04-08 12:15:00	2026-04-08 15:15:00
79	20	1	REIS-1079	Отправлен	2028.00	2026-04-09 16:15:00	2026-04-09 22:15:00
78	7	2	REIS-1078	Прибыл	2215.00	2026-03-14 08:15:00	2026-03-14 12:15:00
77	12	9	REIS-1077	Прибыл	2096.00	2026-03-02 17:00:00	2026-03-03 02:00:00
76	7	8	REIS-1076	Прибыл	2419.00	2026-03-15 13:30:00	2026-03-15 21:30:00
75	16	10	REIS-1075	Прибыл	1581.00	2026-03-17 14:00:00	2026-03-18 00:00:00
74	18	7	REIS-1074	Прибыл	1075.00	2026-03-06 10:45:00	2026-03-06 12:45:00
73	18	3	REIS-1073	Прибыл	3293.00	2026-03-19 20:30:00	2026-03-20 00:30:00

Теперь вызовем процедуру:

```
Vokzal=# CALL "Vokzal".add_trip(
Vokzal(#      7,
Vokzal(#      8,
Vokzal(#      'L999',
Vokzal(#      '2026-04-20 09:00:00',
Vokzal(#      '2026-04-20 15:00:00',
Vokzal(#      2500.00);
ЗАМЕЧАНИЕ: Новый рейс L999 успешно добавлен
CALL
```

Список рейсов после вызова процедуры:

trip_id	route_id	bus_id	number	status	price	departure_date	arrival_date
81	7	8	L999	Запланирован	2500.00	2026-04-20 09:00:00	2026-04-20 15:00:00
80	19	4	REIS-1080	Прибыл	915.00	2026-04-08 12:15:00	2026-04-08 15:15:00
79	20	1	REIS-1079	Отправлен	2028.00	2026-04-09 16:15:00	2026-04-09 22:15:00
78	7	2	REIS-1078	Прибыл	2215.00	2026-03-14 08:15:00	2026-03-14 12:15:00
77	12	9	REIS-1077	Прибыл	2096.00	2026-03-02 17:00:00	2026-03-03 02:00:00
76	7	8	REIS-1076	Прибыл	2419.00	2026-03-15 13:30:00	2026-03-15 21:30:00
75	16	10	REIS-1075	Прибыл	1581.00	2026-03-17 14:00:00	2026-03-18 00:00:00
74	18	7	REIS-1074	Прибыл	1075.00	2026-03-06 10:45:00	2026-03-06 12:45:00

Видим что появился рейс number = 'L999'

Для проверки процедуры **add_trip** были выбраны существующие маршрут и автобус. Перед выполнением процедуры был выведен список последних рейсов. После вызова процедуры в таблице **trip** появилась новая запись с указанным номером рейса, датой отправления, датой прибытия, ценой и статусом **Запланирован**.

2. Создание триггеров

Триггеры, основанные на варианте Автовокзал:

- продажа билетов не раньше чем за 10 суток;
- запрет продажи билетов на рейс со статусом, отличным от Запланирован;
- остановка посадки и остановка высадки не должны совпадать;
- одно место нельзя продать дважды на один рейс;
- место должно принадлежать автобусу выбранного рейса;
- не более 10 билетов без места, скидка 10% для билета без места;
- дата отправления рейса должна быть раньше даты прибытия, цена рейса не может быть отрицательной.

Триггер 1. Проверка периода продажи билета

Билет нельзя купить раньше чем за 10 суток до отправления рейса и нельзя купить после отправления.

```
CREATE OR REPLACE FUNCTION "Vokzal".fn_ticket_purchase_period()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_departure_date timestamp;
BEGIN
    SELECT departure_date
    INTO v_departure_date
    FROM "Vokzal"."trip"
    WHERE trip_id = NEW.trip_id;

    IF v_departure_date IS NULL THEN
        RAISE EXCEPTION 'Для рейса % не найдена дата отправления', NEW.trip_id;
    END IF;

    IF NEW.purchase_date < v_departure_date - INTERVAL '10 days' THEN
        RAISE EXCEPTION 'Билет нельзя купить раньше чем за 10 суток до отправления
рейса';
    END IF;

    IF NEW.purchase_date > v_departure_date THEN
        RAISE EXCEPTION 'Билет нельзя купить после отправления рейса';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_ticket_purchase_period ON "Vokzal"."ticket";
```

```
CREATE TRIGGER trg_ticket_purchase_period
BEFORE INSERT OR UPDATE OF trip_id, purchase_date ON "Vokzal"."ticket"
FOR EACH ROW
EXECUTE FUNCTION "Vokzal".fn_ticket_purchase_period();
```

Сначала создадим рейс через 20 дней. Это специально неверный случай для продажи сейчас.

Пусть route_id = 4
 bus_id = 2
 passenger_id = 1
 stop_id 1 = 3
 stop_id 2 = 5

Добавили рейс:

```
Vokzal=# CALL "Vokzal".add_trip(
Vokzal(#      4,
Vokzal(#      2,
Vokzal(#      'T151'::varchar,
Vokzal(# (NOW() + INTERVAL '20 days')::timestamp,
Vokzal(#      (NOW() + INTERVAL '20 days 6 hours')::timestamp,
Vokzal(#      2500.00::numeric
Vokzal(# );
ЗАМЕЧАНИЕ: Новый рейс T151 успешно добавлен
CALL
```

trip_id	route_id	bus_id	number	status	price	departure_date	arrival_date
82	4	2	T151	Запланирован	2500.00	2026-06-01 21:41:34.728404	2026-06-02 03:41:34.728404

получили trip_id = 82

И при попытке купить билет:

```
Vokzal=# CALL "Vokzal".sell_ticket(
Vokzal(#      82,
Vokzal(#      1,
Vokzal(#      3,
Vokzal(#      5,
Vokzal(#      NULL::integer,
Vokzal(#      'Онлайн',
Vokzal(#      'T151');
ОШИБКА: Билет нельзя купить раньше чем за 10 суток до отправления рейса
```

Триггер **trg_ticket_purchase_period** проверяет дату покупки билета. Для проверки был создан рейс **T151** с датой отправления через 20 дней. После этого была выполнена попытка продажи билета на данный рейс. Так как продажа выполнялась раньше чем за 10 суток до отправления, триггер заблокировал операцию.

Триггер 2. Запрет продажи билета на недопустимый рейс

Запрещает продавать билеты на рейс, если его статус не **Запланирован**

Сперва посмотрим на список рейсов и их статусов:

trip_id	number	status	departure_date
82	T151	Запланирован	2026-06-01 21:41:34.728404
81	L999	Запланирован	2026-04-20 09:00:00
80	REIS-1080	Прибыл	2026-04-08 12:15:00
79	REIS-1079	Отправлен	2026-04-09 16:15:00
78	REIS-1078	Прибыл	2026-03-14 08:15:00
77	REIS-1077	Прибыл	2026-03-02 17:00:00
76	REIS-1076	Прибыл	2026-03-15 13:30:00
75	REIS-1075	Прибыл	2026-03-17 14:00:00
74	REIS-1074	Прибыл	2026-03-06 10:45:00
73	REIS-1073	Прибыл	2026-03-19 20:30:00

```
CREATE OR REPLACE FUNCTION "Vokzal".fn_ticket_trip_available()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_status varchar;
BEGIN
    SELECT status
    INTO v_status
    FROM "Vokzal"."trip"
    WHERE trip_id = NEW.trip_id;

    IF v_status IS NULL THEN
        RAISE EXCEPTION 'Рейс % не найден', NEW.trip_id;
    END IF;

    IF v_status <> 'Запланирован' THEN
        RAISE EXCEPTION 'Продажа билета невозможна. Статус рейса: %', v_status;
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_ticket_trip_available ON "Vokzal"."ticket";

CREATE TRIGGER trg_ticket_trip_available
BEFORE INSERT OR UPDATE OF trip_id ON "Vokzal"."ticket"
FOR EACH ROW
EXECUTE FUNCTION "Vokzal".fn_ticket_trip_available();
```

Создадим рейс в пределах 10 суток:

```
Vokzal=# CALL "Vokzal".add_trip(
Vokzal(# 1,
Vokzal(# 1,
Vokzal(# 'TG102'::varchar,
Vokzal(# (NOW() + INTERVAL '5 days')::timestamp,
Vokzal(# (NOW() + INTERVAL '5 days 6 hours')::timestamp,
Vokzal(# 2500.00::numeric
Vokzal(# );
ЗАМЕЧАНИЕ: Новый рейс TG102 успешно добавлен
```

Теперь меняем статус рейса на отменен:

```
Vokzal=# UPDATE "Vokzal"."trip"
Vokzal=# SET status = 'Отменён'
Vokzal=# WHERE number = 'TG102';
UPDATE 1
```

Пытаемся купить билет и видим:

```
Vokzal=# CALL "Vokzal".sell_ticket(
Vokzal(# 83,
Vokzal(# 1,
Vokzal(# 1,
Vokzal(# 2,
Vokzal(# NULL,
Vokzal(# 'Онлайн',
Vokzal(# 'T102');
ОШИБКА: Продажа билета невозможна. Статус рейса: Отменён
```

Триггер **trg_ticket_trip_available** проверяет статус рейса перед продажей билета. Для проверки был создан рейс **TG102**, затем его статус был изменён на **Отменён**. После попытки продажи билета триггер заблокировал операцию, так как продавать билеты можно только на рейсы со статусом **Запланирован**.

Триггер 3. Проверка разных остановок посадки и высадки

Запрещает создать билет, если остановка посадки совпадает с остановкой высадки

```
CREATE OR REPLACE FUNCTION "Vokzal".fn_ticket_different_stops()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.boarding_stop_id = NEW.dropoff_stop_id THEN
        RAISE EXCEPTION 'Остановка посадки и остановка высадки не могут совпадать';
    END IF;

    RETURN NEW;
END;
$$;
```

```
DROP TRIGGER IF EXISTS trg_ticket_different_stops ON "Vokzal"."ticket";
```

```
CREATE TRIGGER trg_ticket_different_stops
BEFORE INSERT OR UPDATE OF boarding_stop_id, dropoff_stop_id ON
"Vokzal"."ticket"
FOR EACH ROW
EXECUTE FUNCTION "Vokzal".fn_ticket_different_stops();
```

Видим что работает:

```
Vokzal=# CALL "Vokzal".sell_ticket(
Vokzal(#      84,
Vokzal(#      2,
Vokzal(#      3,
Vokzal(#      3,
Vokzal(#      NULL::integer,
Vokzal(#      'Онлайн'::varchar,
Vokzal(#      'T103'::varchar
Vokzal(# );
ОШИБКА:  Остановка посадки и остановка высадки не могут совпадать
```


Триггер 4. Запрет повторной продажи одного места

Запрещает продать одно и то же место дважды за один рейс.

Смотрим на билеты с местами до проверки:

ticket_id	number	trip_id	seat_id	status
1485	T1205	4	123	Возвращён
1484	T862564	80	145	Использован
1483	T673629	80	146	Возвращён
1482	T303941	80	127	Использован
1481	T502537	80	137	Куплен
1480	T857641	80	121	Использован
1479	T674608	80	149	Использован
1478	T229287	80	119	Использован
1477	T742699	80	126	Использован
1476	T634256	80	124	Использован

```
CREATE OR REPLACE FUNCTION "Vokzal".fn_ticket_unique_seat()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.seat_id IS NOT NULL THEN
        IF EXISTS (
            SELECT 1
            FROM "Vokzal"."ticket" t
            WHERE t.trip_id = NEW.trip_id
            AND t.seat_id = NEW.seat_id
            AND t.status IN ('Куплен', 'Использован')
            AND t.ticket_id <> COALESCE(NEW.ticket_id, -1)
        ) THEN
            RAISE EXCEPTION 'Место % уже продано на рейс %', NEW.seat_id,
NEW.trip_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_ticket_unique_seat ON "Vokzal"."ticket";

CREATE TRIGGER trg_ticket_unique_seat
BEFORE INSERT OR UPDATE OF trip_id, seat_id ON "Vokzal"."ticket"
FOR EACH ROW
EXECUTE FUNCTION "Vokzal".fn_ticket_unique_seat();
```

Продаем первый билет:

```
Vokzal=# CALL "Vokzal".sell_ticket(  
Vokzal(# 85,  
Vokzal(# 1,  
Vokzal(# 1,  
Vokzal(# 2,  
Vokzal(# 5,  
Vokzal(# 'Онлайн',  
Vokzal(# 'T104A');  
ЗАМЕЧАНИЕ: Билет T104A успешно продан. Итоговая цена: 2500.00
```

Затем второй пытаемся и видим:

```
Vokzal=# CALL "Vokzal".sell_ticket(  
Vokzal(# 85,  
Vokzal(# 1,  
Vokzal(# 1,  
Vokzal(# 2,  
Vokzal(# 5,  
Vokzal(# 'Онлайн',  
Vokzal(# 'T104A');  
ОШИБКА: Место 5 уже продано на рейс 85
```

Триггер **trg_ticket_unique_seat** проверяет уникальность места на рейсе. Для проверки был создан рейс **TG104**. Сначала был успешно продан билет T104A на выбранное место. Затем была выполнена повторная попытка продать билет T104A на то же место того же рейса. Триггер заблокировал повторную продажу.

Триггер 5. Проверка принадлежности места автобусу рейса

Запрещает продать место из другого автобуса

Посмотрим какие у нас есть места:

seat_id	bus_id	number
1	1	1
2	1	2
3	1	3
4	1	4
5	1	5
6	1	6
7	1	7
8	1	8
9	1	9
10	1	10
11	1	11
12	1	12
13	1	13
14	1	14
15	1	15
16	1	16
17	1	17
18	1	18
19	1	19
20	1	20
21	1	21
22	1	22
23	1	23
24	1	24
25	1	25
26	1	26
27	1	27
28	1	28
29	1	29
30	1	30
31	1	31
32	1	32
33	2	1
34	2	2
35	2	3
36	2	4
37	2	5
38	2	6
39	2	7
40	2	8

```
CREATE OR REPLACE FUNCTION "Vokzal".fn_ticket_seat_bus_match()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_trip_bus_id integer;
    v_seat_bus_id integer;
BEGIN
    IF NEW.seat_id IS NULL THEN
        RETURN NEW;
    END IF;

    SELECT bus_id
    INTO v_trip_bus_id
    FROM "Vokzal"."trip"
    WHERE trip_id = NEW.trip_id;

    SELECT bus_id
    INTO v_seat_bus_id
    FROM "Vokzal"."seat"
    WHERE seat_id = NEW.seat_id;
```

```

IF v_trip_bus_id IS NULL THEN
    RAISE EXCEPTION 'Рейс % не найден', NEW.trip_id;
END IF;

IF v_seat_bus_id IS NULL THEN
    RAISE EXCEPTION 'Место % не найдено', NEW.seat_id;
END IF;

IF v_trip_bus_id <> v_seat_bus_id THEN
    RAISE EXCEPTION 'Место % принадлежит другому автобусу', NEW.seat_id;
END IF;

RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_ticket_seat_bus_match ON "Vokzal"."ticket";

CREATE TRIGGER trg_ticket_seat_bus_match
BEFORE INSERT OR UPDATE OF trip_id, seat_id ON "Vokzal"."ticket"
FOR EACH ROW
EXECUTE FUNCTION "Vokzal".fn_ticket_seat_bus_match();

```

У нас есть рейс:

trip_id	bus_id	number	status
86	1	TG105	Запланирован

У этого рейса bus_id = 1, когда пытаемся продать билет, где seat_id из bus_id = 2, то видим:

```

Vokzal=# CALL "Vokzal".sell_ticket(
Vokzal(# 86,
Vokzal(# 1,
Vokzal(# 1,
Vokzal(# 2,
Vokzal(# 34,
Vokzal(# 'Онлайн',
Vokzal(# 'T105');
ОШИБКА: Место 34 принадлежит другому автобусу

```

Триггер **trg_ticket_seat_bus_match** проверяет, что место в билете принадлежит автобусу, назначенному на данный рейс. Для проверки был создан рейс **TG105**, после чего было выбрано место из другого автобуса. Попытка продажи билета с таким местом была заблокирована.

Триггер 6. Скидка 10% и лимит 10 билетов без места

Если билет без места (seat_id IS NULL), то:

- ставит скидку 10;
- пересчитывает цену как 90% от цены рейса;
- запрещает больше 10 билетов без места на один рейс.

```
CREATE OR REPLACE FUNCTION "Vokzal".fn_ticket_no_seat_limit_discount()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_no_seat_count integer;
    v_base_price numeric(10,2);
BEGIN
    IF NEW.seat_id IS NULL THEN
        SELECT COUNT(*)
        INTO v_no_seat_count
        FROM "Vokzal"."ticket" t
        WHERE t.trip_id = NEW.trip_id
            AND t.seat_id IS NULL
            AND t.status IN ('Куплен', 'Использован')
            AND t.ticket_id <> COALESCE(NEW.ticket_id, -1);

        IF v_no_seat_count >= 10 THEN
            RAISE EXCEPTION 'На рейс % уже продано 10 билетов без места', NEW.trip_id;
        END IF;

        SELECT price
        INTO v_base_price
        FROM "Vokzal"."trip"
        WHERE trip_id = NEW.trip_id;

        IF v_base_price IS NULL THEN
            RAISE EXCEPTION 'Не найдена базовая цена рейса %', NEW.trip_id;
        END IF;

        NEW.discount := 10;
        NEW.price := ROUND(v_base_price * 0.9, 2);
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_ticket_no_seat_limit_discount ON "Vokzal"."ticket";

CREATE TRIGGER trg_ticket_no_seat_limit_discount
```

```
BEFORE INSERT OR UPDATE OF trip_id, seat_id ON "Vokzal"."ticket"
FOR EACH ROW
EXECUTE FUNCTION "Vokzal".fn_ticket_no_seat_limit_discount();
```

До наших действий мы видим стоячие места:

ticket_id	number	trip_id	seat_id	price	discount	status
1472	T733754	79		1825.20	10.00	Куплен
1454	T806017	78		1993.50	10.00	Использован
1453	T257690	78		1993.50	10.00	Использован
1452	T567838	78		1993.50	10.00	Использован
1413	T665200	76		2177.10	10.00	Использован
1412	T435158	76		2177.10	10.00	Использован
1394	T621586	75		1422.90	10.00	Использован
1369	T578091	74		967.50	10.00	Возвращён

Продадим билет:

```
Vokzal=# CALL "Vokzal".sell_ticket(
Vokzal(# 87,
Vokzal(# 1,
Vokzal(# 1,
Vokzal(# 2,
Vokzal(# NULL,
Vokzal(# 'Онлайн',
Vokzal(# 'T106');
ЗАМЕЧАНИЕ: Билет T106 успешно продан. Итоговая цена: 2250.00
```

В списке билетов видим:

ticket_id	number	trip_id	seat_id	price	discount	status
1498	T106	87		2250.00	10.00	Куплен

Пытаясь продать 11 стоячих билетов мы видим:

```
Vokzal=# CALL "Vokzal".sell_ticket(87, 1, 1, 2, NULL, 'Онлайн', 'T106A4');
ЗАМЕЧАНИЕ: Билет T106A4 успешно продан. Итоговая цена: 2250.00
CALL
Vokzal=# CALL "Vokzal".sell_ticket(87, 1, 1, 2, NULL, 'Онлайн', 'T106A5');
ЗАМЕЧАНИЕ: Билет T106A5 успешно продан. Итоговая цена: 2250.00
CALL
Vokzal=# CALL "Vokzal".sell_ticket(87, 1, 1, 2, NULL, 'Онлайн', 'T106A6');
ЗАМЕЧАНИЕ: Билет T106A6 успешно продан. Итоговая цена: 2250.00
CALL
Vokzal=# CALL "Vokzal".sell_ticket(87, 1, 1, 2, NULL, 'Онлайн', 'T106A7');
ЗАМЕЧАНИЕ: Билет T106A7 успешно продан. Итоговая цена: 2250.00
CALL
Vokzal=# CALL "Vokzal".sell_ticket(87, 1, 1, 2, NULL, 'Онлайн', 'T106A8');
ЗАМЕЧАНИЕ: Билет T106A8 успешно продан. Итоговая цена: 2250.00
CALL
Vokzal=# CALL "Vokzal".sell_ticket(87, 1, 1, 2, NULL, 'Онлайн', 'T106A9');
ЗАМЕЧАНИЕ: Билет T106A9 успешно продан. Итоговая цена: 2250.00
CALL
Vokzal=# CALL "Vokzal".sell_ticket(87, 1, 1, 2, NULL, 'Онлайн', 'T106A1');
ОШИБКА: На рейс 87 уже продано 10 билетов без места
```

Триггер **trg_ticket_no_seat_limit_discount** реализует правило продажи билетов без места. При продаже билета без места триггер автоматически установил скидку 10% и пересчитал цену билета. Также был проверен лимит: после продажи 10 билетов без места попытка продажи 11-го билета была заблокирована.

Триггер 7. Проверка корректности данных рейса

Запрещает:

- дату отправления позже или равную дате прибытия;
- отрицательную цену рейса.

```
CREATE OR REPLACE FUNCTION "Vokzal".fn_trip_validate()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.departure_date IS NOT NULL
        AND NEW.arrival_date IS NOT NULL
        AND NEW.departure_date >= NEW.arrival_date THEN
        RAISE EXCEPTION 'Дата отправления должна быть раньше даты прибытия';
    END IF;

    IF NEW.price IS NOT NULL AND NEW.price < 0 THEN
        RAISE EXCEPTION 'Цена рейса не может быть отрицательной';
    END IF;

    RETURN NEW;
END;
$$;
```

```
DROP TRIGGER IF EXISTS trg_trip_validate ON "Vokzal"."trip";
```

```
CREATE TRIGGER trg_trip_validate
BEFORE INSERT OR UPDATE OF departure_date, arrival_date, price ON "Vokzal"."trip"
FOR EACH ROW
EXECUTE FUNCTION "Vokzal".fn_trip_validate();
```

Ошибка по датам:

```
Vokzal=# INSERT INTO "Vokzal"."trip" (
Vokzal(#      route_id,
Vokzal(#      bus_id,
Vokzal(#      number,
Vokzal(#      status,
Vokzal(#      price,
Vokzal(#      departure_date,
Vokzal(#      arrival_date
Vokzal(# )
Vokzal=# VALUES (
Vokzal(#      1,
Vokzal(#      1,
Vokzal(#      'BADDATE',
Vokzal(#      'Запланирован',
Vokzal(#      1000.00,
Vokzal(#      '2026-05-01 15:00:00'::timestamp,
Vokzal(#      '2026-05-01 10:00:00'::timestamp
Vokzal(# );
ОШИБКА:  Дата отправления должна быть раньше даты прибытия
```

Цена не может быть отрицательной:

```
Vokzal=# INSERT INTO "Vokzal"."trip" (  
Vokzal(#      route_id,  
Vokzal(#      bus_id,  
Vokzal(#      number,  
Vokzal(#      status,  
Vokzal(#      price,  
Vokzal(#      departure_date,  
Vokzal(#      arrival_date  
Vokzal(# )  
Vokzal=# VALUES (  
Vokzal(#      1,  
Vokzal(#      1,  
Vokzal(#      'BADPRICE',  
Vokzal(#      'Запланирован',  
Vokzal(#      -1000.00,  
Vokzal(#      '2026-05-01 10:00:00'::timestamp,  
Vokzal(#      '2026-05-01 15:00:00'::timestamp  
Vokzal(# );  
ОШИБКА: Цена рейса не может быть отрицательной
```

Триггер **trg_trip_validate** проверяет корректность данных рейса. Для проверки были выполнены две некорректные вставки: в первой дата отправления была позже даты прибытия, во второй была указана отрицательная цена рейса. В обоих случаях триггер заблокировал добавление записи.

Выводы

В ходе лабораторной работы были созданы и протестированы хранимые процедуры и авторские триггеры для базы данных “Автовокзал”. Процедуры реализуют продажу билета, возврат билета и добавление нового рейса. Триггеры обеспечивают контроль правил предметной области: ограничение периода предварительной продажи билетов, запрет продажи билетов на недоступные рейсы, проверку остановок посадки и высадки, запрет повторной продажи места, проверку принадлежности места автобусу рейса, автоматическую скидку и ограничение количества билетов без места, а также проверку корректности данных рейса